目次

要件定義書: GitNexus Agent Context Graph **2**
1. 文書概要 2
1.1 目的 2
1.2 スコープ 2
1.3 用語定義 2
1.4 参照文書 3
1.5 レビュー履歴 3
2. 現状分析 3
2.1 現在のコンテキスト読み込み方式 3
2.2 問題点 4
2.3 解決方針 4
2.4 目標トークン削減（段階的アプローチ） 4
3. 機能要件 4
FR-001: Agent Graph Indexer 4
FR-002: Context Resolver 7
FR-003: CLI Integration 11
FR-004: SKILL フロントマター規約 12
FR-005: Code-Agent Bridge 13
FR-006: 自動更新 13
FR-007: Claude Code統合（cc-hayashi提案） 13
4. 非機能要件 14
NFR-001: 性能 14
NFR-002: 信頼性 14
NFR-003: 互換性 14
NFR-004: 運用性 15
NFR-005: セキュリティ 15
5. データフロー 15
5.1 インデックスフロー 15
5.2 コンテキスト探索フロー（ハイブリッドスコアリング） 16
5.3 トークン推定ロジック 16
6. Cypher スキーマ定義 17
6.1 ノード作成 17
6.2 エッジ作成（AgentRelation テーブル — CodeRelationとは独立） 18
6.3 探索クエリ 18
7. テスト要件 19
7.1 ユニットテスト 19
7.2 統合テスト 19
7.3 回帰テスト 20
7.4 精度評価テスト（Ground Truth） 20
8. 受入基準 20
8.1 Phase 1 完了基準（Indexer） 20
8.2 Phase 2 完了基準（Context Resolver） 21
8.3 Phase 3 完了基準（Code-Agent Bridge） 21
8.4 Phase 4 完了基準（MCP/Claude Code統合） 21
8.5 最終受入基準 21
9. リスクと対策 21

10. 実装制約 22
10.1 技術的制約 22
10.2 運用制約 23
付録A: レビュー統合サマリー 23
主要な反映事項 23
変更履歴 24

要件定義書: GitNexus Agent Context Graph

文書番号: GNX-ACG-REQ-001 **バージョン:** 2.0.0（最終版） **作成日:** 2026-03-20 **最終更新:** 2026-03-20 **作成者:** Miyabi Agent Society / Claude Code **承認者:** 林 駿甫 (Guardian) **ステータス:** APPROVED（8エージェントレビュー済み）

---

1. 文書概要

1.1 目的

本文書は、GitNexus Stable Ops プロジェクトにおける「Agent Context Graph」拡張機能の要件を定義する。この拡張により、AIエージェントのコンテキストウィンドウを効率的に使用するための グラフベースのコンテキ

1.2 スコープ

項目	範囲
対象システム	GitNexus Stable Ops v1.2.0+
対象リポジトリ	HAYASHI_SHUNSUKE（初期ターゲット）、将来は全21リポ
対象ユーザー	Claude Code、OpenClaw Agent、人間オペレーター
実装言語	Bash (bin/)、Python 3.10+ (lib/)、Cypher (schema/)
バックエンド	LadybugDB（既存GitNexusグラフDB）

1.3 用語定義

用語	定義
Agent	タスクを実行するAIエージェント（しきるん、カエデ等）
Skill	エージェントが使用する能力定義ドキュメント（SKILL/*.md
KnowledgeDoc	ルール・プロジェクト情報等の参照ドキュメント（KNOWLE
DataSource	タスク・メモ等の構造化データ（personal-data/**/*.json）
ExternalService	外部API（GitHub, Discord, Telegram等）
Context Chain	タスクに必要なノードの連鎖（Agent→Skill→Knowledge→D
Context Window	LLMが一度に処理できるトークン数の上限
Token Budget	タスク実行に割り当て可能なトークン数

用語	定義
Ground Truth	コンテキスト探索精度を評価するための正解データセット
Hybrid Score	テキスト関連度・グラフ距離・ノードタイプを統合したスコア

1.4 参照文書

文書	パス
プラン文書	`docs/PLAN-agent-context-graph.md`
統合レビュー	`docs/REVIEW-agent-context-graph.md`
依存関係グラフ	`~/dev/HAYASHI_SHUNSUKE/docs/agent-dependency-graph.md`
既存アーキテクチャ	`docs/architecture.md`
運用手順書	`docs/runbook.md`
エージェント定義	`~/dev/HAYASHI_SHUNSUKE/AGENTS.md`
スキルインデックス	`~/dev/HAYASHI_SHUNSUKE/KNOWLEDGE/skills/_index.`

1.5 レビュー履歴

日付	レビュアー	観点	判定
2026-03-20	dev-architect	アーキテクチャ	承認（条件付き）
2026-03-20	guardian	セキュリティ	承認
2026-03-20	promptpro	プロンプト最適化	承認
2026-03-20	dev-reviewer	コードレビュー	承認（7件反映済み）
2026-03-20	dev-tester	テスト戦略	承認（7件反映済み）
2026-03-20	kotowari-dev	実装（SQLite/Node.js）	承認
2026-03-20	sigma	BM25分析	承認
2026-03-20	cc-hayashi	Claude Code統合	承認

---

2. 現状分析

2.1 現在のコンテキスト読み込み方式

```
Claude Code セッション開始
  |
自動読み込み:
  +-- ~/.claude/CLAUDE.md                      (~3,000 tokens)
  +-- ~/.claude/rules/*.md (12ファイル)         (~8,000 tokens)
  +-- ~/dev/CLAUDE.md                          (~12,000 tokens)
  +-- ~/dev/.claude/rules/*.md (12ファイル)     (~10,000 tokens)
  +-- ~/dev/HAYASHI_SHUNSUKE/CLAUDE.md         (~5,000 tokens)
  +-- ~/dev/HAYASHI_SHUNSUKE/.claude/*.md      (~5,000 tokens)
  合計: ~43,000 tokens（固定オーバーヘッド）
```

2.2 問題点

#	問題	影響
P1	全ルールが毎回読み込まれる	コンテキストの60-70%が固定消費
P2	タスクに無関係なエージェント情報が含まれる	ノイズによる判断精度低下
P3	スキルファイルは必要時に個別Readが必要	探索コスト（どのスキルを読むべきか不明）
P4	エージェント→スキル→ナレッジの依存関係が暗黙的	必要コンテキストの特定に人間の知識が必要
P5	コンテキスト不足時の回復手段がない	セッション途中でのコンテキスト追加が非効率

2.3 解決方針

グラフベースのコンテキスト探索により、タスク記述から必要最小限のコンテキストを 自動的に特定・取得する

2.4 目標トークン削減（段階的アプローチ）

レビュー指摘（dev-reviewer R3）を反映し、段階的な削減目標を設定する。

フェーズ	目標トークン数	削減率	根拠
Phase 1-2	~10,000	75%	情報欠落リスクを最小化しつ
Phase 3	~5,000	88%	フィードバックに基づく精度
Phase 4-5	~2,000-4,000	90-95%	Ground Truth評価で精度70%以上を確

---

3. 機能要件

FR-001: Agent Graph Indexer

概要: エージェント・スキル・ナレッジの構造をグラフDBにインデックスする

FR-001-01: ノード生成

要件ID	要件	優先度
FR-001-01-A	AGENTS.md からAgentノードを生成できること	P0
FR-001-01-B	SKILL/**/*.md からSkillノードを生成できること	P0
FR-001-01-C	KNOWLEDGE/**/*.md からKnowledge-Docノードを生成できること	P0
FR-001-01-D	personal-data/**/*.json からData-Sourceノードを生成できること	P1
FR-001-01-E	外部サービス定義からExternalServiceノードを生成できること	P2

Agentノードの属性:

属性	型	必須	説明
agent_id	string	Yes	英語ID（PK, 例: “conductor”）
name	string	Yes	日本語名（例: “しきるん”）
emoji	string	No	表示絵文字
role	string	Yes	役割（例: “Conductor / Orchestrator”）
society	string	Yes	所属社会（例: “development”）
type	string	Yes	エージェント種別（“local” / “openclaw” / “acp”）
pane_id	string	No	tmux永続ペインID
node_binding	string	No	OpenClawノードバインド先
config_path	string	No	設定ファイルパス
keywords	string[]	Yes	タスクマッチング用キーワード

Skillノードの属性:

属性	型	必須	説明
skill_id	string	Yes	スキル名（PK, 例: “teikan-drafter”）
name	string	Yes	表示名
category	string	Yes	カテゴリ（personal/infra/business/c
path	string	Yes	ファイルパス（SKILL/からの相対パス
description	string	No	概要説明
version	string	No	バージョン
location	string	No	実体ファイルの場所
priority	string	No	P0/P1/P2
tags	string[]	No	分類タグ
keywords	string[]	Yes	コンテキスト探索用キーワード
scripts	string[]	No	実装スクリプトパス

KnowledgeDocノードの属性:

属性	型	必須	説明
doc_id	string	Yes	ドキュメントID（PK）
title	string	Yes	ドキュメント名
path	string	Yes	ファイルパス
category	string	Yes	rules/skills/projects/system
type	string	Yes	ファイルタイプ（markdown/code/pdf）

属性	型	必須	説明
`priority`	string	No	P0/P1/P2
`content_summary`	string	No	FTS5検索用の要約テキスト
`token_estimate`	int	Yes	推定トークン数
`last_modified_date`	string	No	最終更新日

DataSourceノードの属性:

属性	型	必須	説明
`name`	string	Yes	データ名（例: “tasks.json”）
`path`	string	Yes	ファイルパス
`schema_type`	string	Yes	JSON / JSONL / CSV
`is_ssot`	bool	Yes	Single Source of Truthか
`write_cli`	string	No	書き込みCLI（排他ロック）

FR-001-02: エッジ生成　AgentRelation テーブルを CodeRelation テーブルとは**完全に分離**して管理する（レ
C1）。

要件ID	要件	優先度
FR-001-02-A	Agent→Skill の USES_SKILL エッジを生成できること	P0
FR-001-02-B	Skill→KnowledgeDoc の DEPENDS_ON エッジを生成できること	P0
FR-001-02-C	Skill→DataSource の READS_DATA/WRITES_DATA エッジを生成できること	P1
FR-001-02-D	Skill→ExternalService の CALLS_SERVICE エッジを生成できること	P1
FR-001-02-E	Skill→Skill の COMPOSES エッジを生成できること	P1
FR-001-02-F	Agent→Agent の ROUTES_TO エッジを生成できること	P2
FR-001-02-G	Skill→Code の IMPLEMENTS_CODE エッジを生成できること	P2

AgentRelationテーブルの属性（kotowari-dev提案を反映）:

属性	型	必須	説明
`relation_id`	int	Yes	リレーションID（PK, AUTO_INCREMENT）
`source_id`	string	Yes	ソースノードID
`source_type`	string	Yes	ソースノードタイプ
`target_id`	string	Yes	ターゲットノードID
`target_type`	string	Yes	ターゲットノードタイプ
`relation_type`	string	Yes	リレーションタイプ（USES_SKILL等）
`weight`	float	No	エッジ重み（0.0-1.0, デフォルト 1.0）
`timestamp`	string	No	作成日時

FR-001-03: インデックス実行

要件ID	要件	優先度
FR-001-03-A	単一コマンドで全ノード・エッジを構築できること	P0
FR-001-03-B	増分更新（変更ファイルのみ再インデックス）に対応すること	P1
FR-001-03-C	既存コードグラフ（CodeRelation）を破壊しないこと	P0
FR-001-03-D	インデックス結果の統計を出力すること	P1
FR-001-03-E	dry-run モードで変更内容をプレビューできること	P2
FR-001-03-F	トランザクション管理によりインデックス中断時に不整合が生じないこと	P1

FR-001-04: パース堅牢性（dev-reviewer R4 対応）

要件ID	要件	優先度
FR-001-04-A	不正なYAMLフロントマターを検出し、スキップ（継続処理）できること	P0
FR-001-04-B	欠損データ（必須フィールド未設定）をログ出力し、デフォルト値で補完す	P1
FR-001-04-C	バリデーション結果のサマリーをインデックス完了時に表示すること	P1
FR-001-04-D	`--strict` モードでバリデーションエラーを致命的エラーとして扱えること	P2

FR-002: Context Resolver

概要: タスク記述からグラフを探索し、必要最小限のコンテキストを返却する

FR-002-01: 入力インターフェース

要件ID	要件	優先度
FR-002-01-A	自然言語のタスク記述からコンテキストを探索できること	P0
FR-002-01-B	エージェント名を直接指定してコンテキストを取得できること	P0
FR-002-01-C	スキル名を直接指定してコンテキストを取得できること	P0

要件ID	要件	優先度
FR-002-01-D	探索深度（depth）を指定できること（デフォルト: 2）	P1
FR-002-01-E	最大ノード数（top-K）を指定できること（デフォルト: 10）	P1
FR-002-01-F	トークン上限を指定できること（デフォルト: 5000）	P1
FR-002-01-G	タスクタイプ（bugfix/feature/refactor）を指定できること	P2

FR-002-02: ハイブリッドスコアリングアルゴリズム　sigma分析結果を反映し、テキスト関連度・グラフ距離

要件ID	要件	優先度
FR-002-02-A	FTS5によるBM25テキストマッチングでノードを特定できること	P0
FR-002-02-B	ヒットしたノードからBFS（幅優先探索）で隣接ノードを展開できること	P0
FR-002-02-C	ハイブリッドスコアリング関数で最終スコアを算出すること	P0
FR-002-02-D	P0ノード（announce等）を常に高スコアにすること	P1
FR-002-02-E	トークン上限に達したら探索を打ち切ること	P0
FR-002-02-F	FTS5とグラフクエリを分離して実行すること	P0
FR-002-02-G	将来のセマンティック検索（embedding）に対応可能な設計であること	P2

ハイブリッドスコアリング関数（sigma提案）:

```
Final_Score = (w_text * BM25_Score) + (w_graph * GraphScore) + (w_type * TypeWeight)
```

ここで:
```
  BM25_Score: FTS5 BM25スコア（k1=1.2, b=0.75 初期値）
  GraphScore: 1 / (BFS_Depth + 1)  ... 近いノードほど高スコア
  TypeWeight: ノードタイプ別の基本重み
```

デフォルト重み:
```
  w_text = 0.5    （テキスト関連度）
  w_graph = 0.3   （グラフ距離）
  w_type = 0.2    （ノードタイプ）
```

ノードタイプ別基本重み:
```
  Agent = 0.9
  Skill = 1.0
  KnowledgeDoc = 0.7
  DataSource = 0.5
  ExternalService = 0.3
```

BM25パラメータ最適化方針（sigma分析結果）:

パラメータ	初期値	調整基準
k1	1.2	Skillノード（短文）が多い場合は現状維持、1.5 に上げる
b	0.75	Skillの簡潔さを重視するなら 0.9-1.0 に上げる

調整はGround Truthデータセットに基づくA/Bテストで実施（Phase 2以降）。

FR-002-03: 出力フォーマット

要件ID	要件	優先度
FR-002-03-A	JSON形式で結果を返却すること	P0
FR-002-03-B	files_to_read フィールドに読むべきファイルパスを含むこと	P0
FR-002-03-C	estimated_tokens フィールドで推定トークン数を返すこと	P0
FR-002-03-D	context_chain フィールドで探索経路を返すこと	P1
FR-002-03-E	savings_vs_full フィールドで削減率を返すこと	P2
FR-002-03-F	構造化Markdownフォーマットでの出力をサポートすること	P1

出力JSON仕様:

```
{
  "version": "2.0",
  "query": "string -- 入力クエリ",
  "task_type": "string | null -- bugfix/feature/refactor",
  "matched_agents": ["string[] -- マッチしたエージェント名"],
  "matched_skills": ["string[] -- マッチしたスキル名"],
  "context_chain": [
    {
      "type": "Agent | Skill | KnowledgeDoc | DataSource | ExternalService",
      "name": "string -- ノード名",
      "path": "string | null -- ファイルパス",
      "score": "float -- 0.0-1.0 のハイブリッドスコア",
      "score_breakdown": {
        "text": "float -- BM25スコア成分",
        "graph": "float -- グラフ距離スコア成分",
        "type": "float -- ノードタイプスコア成分"
      },
      "depth": "int -- 起点ノードからの距離",
      "token_estimate": "int -- 推定トークン数"
    }
  ],
```

```
  "files_to_read": ["string[] -- 読むべきファイルの絶対パス"],
  "estimated_tokens": "int -- 総推定トークン数",
  "savings_vs_full": "string -- 全読み込みとの削減率（例: 95.1%）",
  "metadata": {
    "search_depth": "int",
    "total_nodes_explored": "int",
    "execution_time_ms": "int",
    "scoring_weights": {
      "w_text": "float",
      "w_graph": "float",
      "w_type": "float"
    },
    "bm25_params": {
      "k1": "float",
      "b": "float"
    }
  }
}
```

構造化Markdown出力仕様（cc-hayashi提案、`--format markdown` 指定時）：

```
# Context: {query}

## Matched Agents
- {agent_name} ({role}) -- Score: {score}

## Relevant Skills
### {skill_name}
- Path: {path}
- Keywords: {keywords}
- Score: {score}

## Knowledge Documents
### {doc_name}
- Path: {path}
- Tokens: {token_estimate}

## Data Sources
- {data_name}: {path} (SSOT: {is_ssot})

## Metadata
- Estimated tokens: {total}
- Savings: {savings_vs_full}
- Execution time: {time_ms}ms
```

FR-002-04: フォールバック機構（cc-hayashi提案）

要件ID	要件	優先度
FR-002-04-A	コンテキスト取得失敗時にP1フォールバック（汎用コンテキスト）を返すこと	
FR-002-04-B	結果が空の場合、P0スキル（announce等）のみを含む最小コンテキストを	P0
FR-002-04-C	コンテキスト不足検知時の再取得（depth拡大）をサポートすること	P2

FR-003: CLI Integration

概要: 既存の `gni` CLIに Agent Context Graph コマンドを追加する

要件ID	要件	優先度
FR-003-01	`gni agent-context "クエリ"` で探索を実行できること	P0
FR-003-02	`gni agent-context --agent "法務"` でエージェント指定できること	P0
FR-003-03	`gni agent-context --skill "teikan-drafter"` でスキル指定できること	P0
FR-003-04	`gni agent-context --depth N` で深度指定できること	P1
FR-003-05	`gni agent-context --max-tokens N` でトークン上限指定できること	P1
FR-003-06	`gni agent-context --format json|markdown` で出力形式を選択できること	P1
FR-003-07	`gni agent-context --task-type bug-fix|feature|refactor` でタスクタイプを指定できること	P2
FR-003-08	`gni agent-index` でインデックスを実行できること	P0
FR-003-09	`gni agent-index --dry-run` でプレビューできること	P2
FR-003-10	`gni agent-status` でAgent Graphの統計を表示できること	P1
FR-003-11	`gni agent-list` で全エージェントとスキルを一覧できること	P1

要件ID	要件	優先度
FR-003-12	人間可読なカラー出力と`--` `json`オプションの両対応	P1

FR-004: SKILL フロントマター規約

概要: SKILLファイルに構造化メタデータを追加する

要件ID	要件	優先度
FR-004-01	YAMLフロントマター（`---`区切り）をパースできること	P0
FR-004-02	フロントマターがないSKILLも処理できること（ファイル名から推定）	P0
FR-004-03	`keywords`フィールドをContext Resolverのマッチングに使用すること	P0
FR-004-04	`agents`フィールドからUSES_SKILLエッジを生成すること	P0
FR-004-05	`depends_on`フィールドからDEPENDS_ONエッジを生成すること	P1
FR-004-06	既存58 SKILLファイルへのフロントマター一括追加スクリプトを提供すること	P1

フロントマター仕様:

```
---
# 必須フィールド
name: string         # スキル名（ファイル名と一致推奨）
category: string     # personal | infra | business | content | communication | opencla
keywords: string[]   # コンテキスト探索用キーワード（日本語・英語混在可）

# 推奨フィールド
agents: string[]     # このスキルを使用するエージェント名
depends_on: string[] # 依存するナレッジドキュメント
priority: string     # P0 | P1 | P2

# オプションフィールド
version: string      # セマンティックバージョン
description: string # 概要説明（FTS5検索対象）
reads: string[]      # 読み取るデータソースパス
writes: string[]     # 書き込むデータソースパス
external_services: string[]  # 呼び出す外部サービス名
```

```yaml
scripts: string[]    # 実装スクリプトパス
---
```

FR-005: Code-Agent Bridge

概要: スキルと実体コードを接続する

要件ID	要件	優先度
FR-005-01	Skill→Code の IMPLEMENTS_CODE エッジを生成できること	P2
FR-005-02	scripts/ 配下のスクリプトを対応するSkillに紐付けること	P2
FR-005-03	既存GitNexusのコードシンボルとのMERGEが可能なこと	P2

FR-006: 自動更新

概要: Agent Graphを最新状態に保つ自動化

要件ID	要件	優先度
FR-006-01	SKILL/*.md 変更時にAgent Graphを自動再インデックスすること	P1
FR-006-02	AGENTS.md 変更時にAgent Graphを自動再インデックスすること	P1
FR-006-03	git post-commit フックで変更検知すること	P1
FR-006-04	既存の gitnexus-auto-reindex.sh と共存すること	P0
FR-006-05	cron 対応のバッチ再インデックススクリプトを提供すること	P2

FR-007: Claude Code統合（cc-hayashi提案）

概要: gni agent-context の出力をClaude Codeセッションに統合する

要件ID	要件	優先度
FR-007-01	`gni agent-context` の出力をClaude Codeのプロンプトに直接埋め込めること	P1
FR-007-02	タスクタイプに応じたコンテキスト取得スコープの自動調整	P2
FR-007-03	コンテキスト不足時の動的拡張（再実行による追加取得）をサポートするこ	P3
FR-007-04	構造化出力にメタデータ（ファイルパス、行番号、コミットハッシュ）を含	P2

---

4. 非機能要件

NFR-001: 性能

要件ID	要件	目標値
NFR-001-01	Context Resolverの応答時間	< 3秒
NFR-001-02	Agent Graphインデックス時間（全構築）	< 30秒
NFR-001-03	Agent Graphインデックス時間（増分更新）	< 5秒
NFR-001-04	グラフDBの追加ディスク使用量	< 100MB
NFR-001-05	同時実行時（複数エージェント同時クエリ）のスループット	> 5 req/sec

NFR-002: 信頼性

要件ID	要件	目標値
NFR-002-01	コンテキスト探索の適合率（top-3に正解含む）	> 70%
NFR-002-02	インデクサのエラー率	< 1%
NFR-002-03	既存コードグラフへの影響	ゼロ（非破壊）
NFR-002-04	Ground Truth評価のMean Average Precision (MAP)	> 0.65
NFR-002-05	パース失敗時のグレースフルデグレード	必須（継続処理）

NFR-003: 互換性

要件ID	要件
NFR-003-01	GitNexus Stable Ops v1.2.0+ と互換であること
NFR-003-02	LadybugDB バックエンドに対応すること
NFR-003-03	既存の gni コマンドの動作を変更しないこと

要件ID	要件
NFR-003-04	macOS (arm64) および Linux (x86_64) で動作すること
NFR-003-05	Python 3.10+ で動作すること
NFR-003-06	bash 5.0+ で動作すること
NFR-003-07	better-sqlite3（Node.js）との共存が可能なこと

NFR-004: 運用性

要件ID	要件
NFR-004-01	gitnexus-doctor.sh で Agent Graph の健全性をチェックできること
NFR-004-02	gitnexus-smoke-test.sh で Agent Graph の疎通確認ができること
NFR-004-03	ログ出力は既存のログローテーション機構と統合するこ
NFR-004-04	dry-run モードで実際のD-B変更なしにテスト可能なこと
NFR-004-05	Cypher EXPLAIN による実行計画の確認ができること

NFR-005: セキュリティ

要件ID	要件
NFR-005-01	personal-data/ 内のファイル内容をグラフD-Bに格納しないこと（パスのみ）
NFR-005-02	APIキー・トークンをノード属性に含めないこと
NFR-005-03	PRIVATE/ ディレクトリのファイルをインデックス対象外とするこ
NFR-005-04	.env, credentials, secrets を含むファイルをインデックス対象外とすること
NFR-005-05	コンテキスト出力にファイル内容を含めないこと（パス

---

5. データフロー

5.1 インデックスフロー

```
入力ソース                    パーサ                    出力
                          （バリデーション付き）
AGENTS.md          -->  agent_parser()      -->  Agent nodes (13)
SKILL/**/*.md      -->  skill_parser()      -->  Skill nodes (71)
                         + frontmatter
                           validation
```

```
KNOWLEDGE/**/*.md    -->  knowledge_parser()   -->  KnowledgeDoc nodes (95)
personal-data/**     -->  data_parser()        -->  DataSource nodes (18)
(定義ファイル)      -->  service_parser()    -->  ExternalService nodes (11)
                                                          |
依存関係解析          -->  edge_resolver()      -->  AgentRelation テーブル
                                                      (CodeRelationとは独立)
                                                          |
                        Cypher INSERT        -->  .gitnexus/lbug (グラフDB)
                          (トランザクション内)
```

5.2 コンテキスト探索フロー（ハイブリッドスコアリング）

```
入力: "定款を修正して"
    |
[1] Tokenize & Keyword Extract
    --> keywords: ["定款", "修正"]
    |
[2] FTS5 BM25 テキスト検索（グラフクエリとは分離）
    MATCH (s:Skill) WHERE FTS5_MATCH(s.keywords, $keyword)
    --> text_hits: [teikan-drafter (bm25: 0.95)]
    |
[3] BFS グラフ探索 from text_hits (depth=2):
    --> teikan-drafter --> 法務 Agent (d=1)
    --> teikan-drafter --> K_PROJECTS (d=1)
    --> teikan-drafter --> touki-guide (COMPOSES, d=1)
    --> 法務 --> tracker Skill (d=2)
    |
[4] Hybrid Scoring:
    teikan-drafter: 0.5*0.95 + 0.3*1.0 + 0.2*1.0 = 0.975
    法務 Agent:     0.5*0.0  + 0.3*0.5 + 0.2*0.9 = 0.33
    K_PROJECTS:     0.5*0.0  + 0.3*0.5 + 0.2*0.7 = 0.29
    touki-guide:    0.5*0.2  + 0.3*0.5 + 0.2*1.0 = 0.45
    --> Ranked: teikan(0.975), touki(0.45), 法務(0.33), K_PROJ(0.29)
    |
[5] Token Budget Check:
    --> teikan-drafter.md: ~800 tokens
    --> touki-guide.md: ~600 tokens
    --> PROJECTS/miyabi-llc-setup.md: ~700 tokens
    --> 合計: 2,100 tokens (< 5,000 budget)
    |
[6] Output JSON + Markdown
```

5.3 トークン推定ロジック

```python
def estimate_tokens(file_path: str) -> int:
    """ファイルのトークン数を推定"""
    size = os.path.getsize(file_path)
    # 日英混在ドキュメントの平均係数
```

```
    BYTES_PER_TOKEN = 3.7
    return int(size / BYTES_PER_TOKEN)
```

---

6. Cypher スキーマ定義

6.1 ノード作成

```
-- Agent ノード
CREATE (a:Agent {
    agent_id: 'legal',
    name: '法務',
    emoji: '□ ',
    role: 'Legal Agent',
    society: 'legal',
    type: 'local',
    pane_id: '',
    keywords: ['法務', '定款', '登記', '届出', '法人']
})

-- Skill ノード
CREATE (s:Skill {
    skill_id: 'teikan-drafter',
    name: 'teikan-drafter',
    category: 'business',
    path: 'SKILL/business/teikan-drafter.md',
    priority: 'P1',
    keywords: ['定款', '事業目的', '社員総会', '出資', '代表社員']
})

-- KnowledgeDoc ノード
CREATE (k:KnowledgeDoc {
    doc_id: 'miyabi-llc-setup',
    title: 'miyabi-llc-setup',
    path: 'PROJECTS/miyabi-llc-setup.md',
    category: 'projects',
    type: 'markdown',
    priority: 'P0',
    token_estimate: 700
})

-- DataSource ノード
CREATE (d:DataSource {
    name: 'tasks.json',
    path: 'personal-data/tasks/tasks.json',
    schema_type: 'JSON',
    is_ssot: true,
    write_cli: 'AGENT/task-sync.sh'
```

```
})
```

6.2 エッジ作成（AgentRelation テーブル — CodeRelationとは独立）

```
-- Agent --> Skill (USES_SKILL)
CREATE (:Agent {agent_id:'legal'})
  -[:AgentRelation {type:'USES_SKILL', weight:1.0}]->
  (:Skill {skill_id:'teikan-drafter'})

-- Skill --> KnowledgeDoc (DEPENDS_ON)
CREATE (:Skill {skill_id:'teikan-drafter'})
  -[:AgentRelation {type:'DEPENDS_ON', weight:0.9}]->
  (:KnowledgeDoc {doc_id:'miyabi-llc-setup'})

-- Skill --> DataSource (READS_DATA / WRITES_DATA)
CREATE (:Skill {skill_id:'task-tracker'})
  -[:AgentRelation {type:'READS_DATA'}]->
  (:DataSource {name:'tasks.json'})

-- Skill --> Skill (COMPOSES)
CREATE (:Skill {skill_id:'voice-bridge'})
  -[:AgentRelation {type:'COMPOSES'}]->
  (:Skill {skill_id:'announce'})

-- Agent --> Agent (ROUTES_TO)
CREATE (:Agent {agent_id:'conductor'})
  -[:AgentRelation {type:'ROUTES_TO'}]->
  (:Agent {agent_id:'kaede'})
```

6.3 探索クエリ

```
-- タスクキーワードから関連コンテキストチェーンを取得
MATCH (s:Skill)
WHERE ANY(kw IN s.keywords WHERE kw CONTAINS $keyword)
WITH s ORDER BY size([kw IN s.keywords WHERE kw CONTAINS $keyword]) DESC
LIMIT 5
OPTIONAL MATCH (a:Agent)-[:AgentRelation {type:'USES_SKILL'}]->(s)
OPTIONAL MATCH (s)-[:AgentRelation {type:'DEPENDS_ON'}]->(k:KnowledgeDoc)
OPTIONAL MATCH (s)-[:AgentRelation {type:'READS_DATA'}]->(d:DataSource)
OPTIONAL MATCH (s)-[:AgentRelation {type:'COMPOSES'}]->(s2:Skill)
RETURN
  a.name AS agent,
  a.role AS agent_role,
  s.name AS skill,
  s.path AS skill_path,
  s.keywords AS skill_keywords,
  collect(DISTINCT {name: k.title, path: k.path, tokens: k.token_estimate})
    AS knowledge,
```

```
  collect(DISTINCT {name: d.name, path: d.path}) AS data_sources,
  collect(DISTINCT {name: s2.name, path: s2.path}) AS composed_skills
```

---

7. テスト要件

7.1 ユニットテスト

テストID	対象	検証内容
UT-001	agent_parser	AGENTS.md から正しくAgent-tノードが生成されること
UT-002	skill_parser	フロントマター付きSKILLが正しくパースされる
UT-003	skill_parser	フロントマターなしSKILLがファイル名から推定
UT-004	skill_parser	不正フロントマターがスキップされ継続処理され
UT-005	knowledge_parser	KNOWLEDGE/配下のMDがKnowledgeDocノー
UT-006	edge_resolver	Agent→Skill の USES_SKILL エッジが正しく生成されること
UT-007	hybrid_scorer	ハイブリッドスコアが正しく計算されること
UT-008	context_resolver	キーワード “定款” で teikan-drafter がヒットすること
UT-009	context_resolver	depth=2 で隣接ノードが展開されること
UT-010	context_resolver	token_budget 超過時に探索が打ち切られること
UT-011	token_estimator	ファイルサイズからトークン数が概算されること
UT-012	output_format	JSON出力が仕様v2.0に準拠すること
UT-013	output_format	Markdown出力が構造化仕様に準拠すること
UT-014	fallback	結果が空の場合にP0最小コンテキストが返ること
UT-015	validation	バリデーションエラーのサマリーが出力されるこ

7.2 統合テスト

テストID	シナリオ	期待結果
IT-001	“定款を修正して”	法務Agent, teikan-drafter, K_PROJECTS がチェーンに含まれる
IT-002	“タスクを追加して”	パーソナルAgent, task-tracker, tasks.json がチェーンに含まれる
IT-003	“PRを作成して”	ツバキAgent, githubops-workflow, K_RULES がチェーンに含まれる

テストID	シナリオ	期待結果
IT-004	“健康データを分析して”	医療Agent, pushcut, D_HEALTH がチェーンに含まれる
IT-005	“GitNexusをreindexして”	(Agent無し), gitnexus-cli, K_PROJECTS がチェーンに含まれる
IT-006	depth=1 指定	直接接続ノードのみ返却される
IT-007	max-tokens=1000 指定	1000トークン以内で打ち切られる
IT-008	存在しないキーワード	フォールバック（P0最小コンテキスト）が返
IT-009	–format markdown	構造化Markdown形式で出力される
IT-010	並行クエリ（5件同時）	全クエリが3秒以内に応答する

7.3 回帰テスト

テストID	検証内容
RT-001	Agent Graph インデックス後、既存コードグラフのクエリが正常に動作する
RT-002	gni impact / gni context / gni cypher が変更なく動作すること
RT-003	gitnexus-smoke-test.sh が全パスすること
RT-004	CodeRelation テーブルのレコード数がインデックス前後で変化しないこと

7.4 精度評価テスト（Ground Truth）

テストID	検証内容
GT-001	Ground Truthデータセット（10件以上）を構築すること
GT-002	各クエリに対してPrecision@3を計測し、平均 > 70% を確認すること
GT-003	MAP (Mean Average Precision) > 0.65 を確認すること
GT-004	BM25パラメータ変更前後の精度比較を実施すること

---

8. 受入基準

8.1 Phase 1 完了基準（Indexer）

- □ Agent Graph Indexer が HAYASHI_SHUNSUKE リポジトリで動作する
- □ 179+ ノード（13 Agent + 71 Skill + 95 Knowledge）が生成される
- □ 127+ エッジが AgentRelation テーブルに生成される

- ☐ CodeRelation テーブルが影響を受けない（RT-004パス）
- ☐ 不正フロントマターがスキップされ処理が継続する（UT-004パス）
- ☐ UT-001 – UT-006, UT-015 が全パス

8.2 Phase 2 完了基準（Context Resolver）

- ☐ `gni agent-context "定款を修正して"` が正しい結果を返す
- ☐ ハイブリッドスコアリング関数が動作する
- ☐ 応答時間 < 3秒
- ☐ IT-001 – IT-010 が全パス
- ☐ JSON出力仕様v2.0に準拠
- ☐ Ground Truthデータセット10件以上を構築（GT-001）
- ☐ コンテキスト削減率 > 75%

8.3 Phase 3 完了基準（Code-Agent Bridge）

- ☐ Skill→Code の IMPLEMENTS_CODE エッジが生成される
- ☐ コード変更のインパクトがエージェント層まで追跡可能
- ☐ コンテキスト削減率 > 88%

8.4 Phase 4 完了基準（MCP/Claude Code統合）

- ☐ Claude Code から gni agent-context を呼び出せる
- ☐ 構造化Markdown出力が動作する
- ☐ フォールバック機構が動作する
- ☐ コンテキスト削減率 > 90%

8.5 最終受入基準

- ☐ 全テスト（UT + IT + RT + GT）がパス
- ☐ コンテキスト削減率 > 90%（平均）
- ☐ Precision@3 > 70%（Ground Truth評価）
- ☐ MAP > 0.65
- ☐ gitnexus-doctor で Agent Graph が healthy
- ☐ ドキュメント（runbook, architecture）が更新されている

---

9. リスクと対策

#	リスク	影響	確率	指摘元	対策
R1	GitNexus の Cypher INSERT が Agent/Skill 型をサポートしない	Phase 1 ブロック	MEDIUM	dev-architect	Community ノード拡張でフォールバック FTS5 別DB
R2	95% トークン削減で必要な情報が欠落	判断精度低下	HIGH	dev-reviewer, cc-hayashi	段階的削減（75%→88%→9 95%）、フィードバックルー

#	リスク	影響	確率	指摘元	対策
R3	キーワードマッチの精度が低い	検索体験低下	MEDIUM	sigma	Ground Truth評価 + BM25パラメータA/Bテスト + 辞書作成
R4	Agent Graph が既存コードグラフと干渉	運用障害	LOW	dev-architect	AgentRelation テーブル完全分離（合意事項
R5	LadybugDB のスキーマ制約でカスタムノード型不可	設計変更	MEDIUM	dev-architect	Community ノードの type 属性で Agent/Skill を表現
R6	SKILLフロントマターのバリデーション不備	品質劣化	MEDIUM	dev-reviewer, kotowari-dev	バリデーション強化 + エラーハンドリング + –strict モード
R7	SQLite書き込みロック競合	同時実行時のエラー	LOW	kotowari-dev	バッチ処理 + トランザクション管理 + better-sqlite3
R8	メタデータからの機密情報漏洩	セキュリティ	LOW	guardian	パスのみ格納ポリシー + PRIVATE除外 + .env除外
R9	テストカバレッジ不足	品質劣化	MEDIUM	dev-tester	Ground Truth構築 + CI/CD組込 + Precision/MAP計測

---

10. 実装制約

10.1 技術的制約

#	制約	理由
TC-1	FTS5はテキスト検索専用、グラフ走査はCypher推奨	kotowari-dev推奨: パフォーマンス・保守性
TC-2	AgentRelation と CodeRelation は物理的に別テーブル	合意事項C1: 全レビュアー一致
TC-3	ノードにファイル内容を格納しない（パスのみ）	NFR-005: セキュリティ要件
TC-4	better-sqlite3 を使用する場合は同期API前提	kotowari-dev推奨: 安定性

#	制約	理由
TC-5	BM25パラメータはGround Truth構築後にのみ変更	sigma推奨: 根拠なき変更禁止

10.2 運用制約

#	制約	理由
OC-1	既存の gni コマンドの動作を変更しない	NFR-003: 後方互換性
OC-2	gitnexus-auto-reindex.sh と共存する	FR-006-04
OC-3	Node.js v24+ が必要（FTS5サポート）	既知の制約: CLAUDE.md記載

付録A: レビュー統合サマリー

本文書はv1.0.0（DRAFT）を基に、OpenClaw MAS 8エージェントのレビュー結果を統合して作成された。

主要な反映事項

#	反映事項	指摘元	反映箇所
1	AgentRelation テーブル分離を明示化	dev-architect, dev-reviewer, kotowari-dev	FR-001-02, 6.2
2	ハイブリッドスコアリング関数の追加	sigma	FR-002-02-C
3	段階的トークン削減目標の設定	dev-reviewer	2.4
4	パース堅牢性要件の追加（FR-001-04）	dev-reviewer, kotowari-dev	FR-001-04
5	フォールバック機構の追加（FR-002-04）	cc-hayashi	FR-002-04
6	Claude Code統合要件の追加（FR-007）	cc-hayashi	FR-007
7	Ground Truth評価テストの追加	dev-tester, sigma	7.4
8	セキュリティ要件の強化	guardian	NFR-005
9	出力仕様にscore_breakdown追加	sigma	FR-002-03
10	構造化Markdown出力の追加	cc-hayashi	FR-002-03-F
11	ノード属性の詳細化（type, description等）	kotowari-dev	FR-001-01
12	並行クエリの性能要件追加	dev-tester	NFR-001-05
13	技術的制約の明文化	kotowari-dev, sigma	10.1

#	反映事項	指摘元	反映箇所
14	リスクマトリクスの拡充（R6-全レビュアー R9追加）		9

変更履歴

日付	バージョン	変更内容	作成者
2026-03-20	1.0.0	初版作成	Claude Code
2026-03-20	2.0.0	8エージェントレビュー結果を統合。ハイブリッドスコアリング不採用。Code統合・Ground Truth評価を追加、セキュリティ強化、リスクマトリクス拡充	Claude Code